

Learning a Neural Calculator: Comparing a CNN Regressor and a Transformer Classifier

Group Members: Name Surname, Name Surname, Name Surname

Abstract

This project studies whether neural networks can learn to evaluate symbolic arithmetic expressions from supervised examples. Each input is a string composed of digits, addition, subtraction and parentheses, and the target is the integer value of the expression. The main difficulty is not only in-distribution prediction, but also generalization to out-of-distribution formula shapes and longer expressions. We compare two substantially different neural approaches: a convolutional regressor, which directly predicts a real-valued numerical output, and a Transformer classifier, which predicts one of the possible integer result classes. The CNN regressor achieves the strongest results overall, with high in-distribution accuracy and substantially better long-sequence extrapolation. The Transformer classifier also performs well on validation and ID data, but degrades more sharply on the long-sequence split. These results suggest that, for this task, a carefully trained convolutional regression model can provide a strong balance between exact-match accuracy and numerical stability.

1 Introduction

The goal of this project is to train a neural model that receives an arithmetic expression as a string and predicts its integer output value. Expressions are built from single decimal digits, the binary operators $+$ and $-$, and parentheses. Although the grammar is simple, the task is non-trivial for neural networks because the model must learn order-sensitive and compositional rules from examples rather than using a symbolic evaluator.

The main objective of the full practical track is generalization beyond the training distribution. A successful model should not only perform well on in-distribution examples, but should also be robust to unusual expression structures, heavier use of parentheses, different subtraction patterns and longer sequences. We therefore compare two different modelling strategies. The first is a one-dimensional convolutional neural network used as a regressor. This model learns local arithmetic patterns and outputs a real-valued prediction. The second is a Transformer encoder used as a classifier over the 199 possible integer outputs from -99 to 99 . This model uses self-attention to represent global relations between tokens and predicts a discrete integer class.

The comparison is not intended as a pure architectural ablation, because the two models differ both in architecture and output formulation. Instead, we compare two complete neural approaches to the same task: continuous numerical approximation through convolutional regression versus discrete integer prediction through attention-based classification.

2 Data and Evaluation Protocol

The project provides fixed data splits: a training set of 12,000 examples, a validation set of 2,000 examples, an in-distribution test set of 2,000 examples, an out-of-distribution test set of 2,000 examples, and a long-sequence test set of 1,500 examples. Each example contains the expression, its integer value, the expression length, the number of operators and the parenthesis depth.

The validation and ID splits have a distribution very similar to the training set. In the training set, the average expression length is approximately 9.61, the average operator count is 3.49, and the average depth is 3.69. The OOD split is structurally more difficult, with average length 12.48, average operator count 4.52 and average depth 4.43. The long-sequence split is the hardest distribution shift: its average length is 17.74, its average operator count is 6.21 and its average depth is 5.50. Moreover, 95.27% of long-sequence examples contain parentheses, compared with 61.96% in the training set. This confirms that the long split tests not only length extrapolation, but also increased structural complexity.

All models are evaluated using exact-match accuracy, mean absolute error (MAE) and root mean squared error (RMSE). Exact-match accuracy is computed after rounding the numerical prediction to the nearest integer. MAE and RMSE are computed on the raw numerical predictions. For the classifier, the raw numerical prediction is the integer value corresponding to the predicted class. For the regressor, the raw numerical prediction is the scalar output after denormalization.

The test sets are used only for final evaluation. Hyperparameters, early stopping, learning-rate scheduling and checkpoint selection are based on validation performance.

3 Models

3.1 Shared Input Representation

Both models use character-level tokenization. Each expression is represented as a sequence of tokens corresponding to the digits, the two operators, the parentheses and a padding symbol. Character tokenization is appropriate for this task because the grammar itself is defined over individual characters and all numbers are single decimal digits.

Sequences have different lengths, so mini-batches are padded. Padding is used only to construct equal-length tensors and is not treated as meaningful input. The models therefore use masking or masked readout operations so that padded positions do not influence the final expression representation.

3.2 CNN Regressor

The CNN model treats the arithmetic expression as a one-dimensional sequence. Its architecture is:

input characters $\rightarrow$ one-hot representation $\rightarrow$ Conv1D blocks $\rightarrow$ masked readout $\rightarrow$ linear regression head.

The convolutional layers apply shared local filters across the expression. This is motivated by the fact that many arithmetic structures contain repeated local patterns, such as digit-operator-digit fragments or parenthesized subexpressions. We avoid striding and aggressive intermediate pooling because they may discard order-sensitive information. After the convolutional blocks, the model aggregates the sequence representation into a fixed-size vector. The final layer is linear and outputs a single scalar.

The regression target is normalized using the mean and standard deviation computed on the training set. During evaluation, predictions are denormalized before computing exact-match accuracy, MAE and RMSE. The final selected CNN uses a custom regression loss designed to combine stable numerical optimization with the exact-match objective. The loss includes a regression component on normalized targets and an additional penalty encouraging denormalized predictions to lie within the correct rounding interval. This was introduced because standard regression losses can give low numerical error while still failing exact-match accuracy near rounding boundaries.

The best CNN checkpoint was selected at epoch 86, with validation exact-match accuracy 98.75%, validation RMSE 0.1425 and validation MAE 0.0938.

3.3 Transformer Classifier

The Transformer model uses an encoder-only architecture. Its architecture is:

input characters $\rightarrow$ token embeddings + sinusoidal positional encoding $\rightarrow$ Transformer encoder $\rightarrow$ sequence readout $\rightarrow$ 199-class

The Transformer is motivated by the need to model global dependencies between distant tokens. Parentheses and subtraction can make the effect of a token depend on non-local parts of the expression, so self-attention is a natural alternative to purely local convolutional filters.

The model uses sinusoidal positional encodings rather than learned positional embeddings. This choice is important because the long-sequence test contains positions and lengths that are less represented in training; fixed positional encodings are therefore more appropriate for length extrapolation.

The final Transformer configuration uses embedding dimension 128, 4 attention heads, 3 encoder layers, feed-forward dimension 512, dropout 0.1 and GELU activation. A mean-plus-max sequence readout is used: masked mean pooling summarizes the full expression, while masked max pooling preserves strong token-level activations that might otherwise be diluted in longer expressions. The two pooled vectors are concatenated and passed to a linear classifier with 199 outputs, corresponding to the values from -99 to 99 .

The Transformer is trained using cross-entropy loss. Since the target values lie in $[-99, 99]$, each value y is mapped to the class index $y + 99$. At evaluation time, the predicted class is converted back into its corresponding integer value.

4 Training Procedure and Model Selection

Both models are trained using Adam with learning rate 0.001 and batch size 128. Validation exact-match accuracy is the primary criterion for model selection, because the project thresholds are based on exact integer prediction. Validation MAE and RMSE are used as secondary indicators of numerical quality.

A learning-rate scheduler is used to reduce the learning rate when validation accuracy reaches a plateau. Early stopping is also based on validation accuracy, preventing unnecessary training once validation performance no longer improves. The final model for each architecture is the best validation checkpoint, not necessarily the last epoch.

This protocol is important because the validation set is used dynamically during training for checkpoint selection, patience and learning-rate scheduling. Therefore, the final models are trained on the training split only, while the validation split remains an independent model-selection signal. The ID, OOD and long-sequence test sets are evaluated only after the model and hyperparameters have been fixed.

For the Transformer, additional focused training was used to improve robustness to longer and more difficult expressions. Hard examples from the training distribution were oversampled, especially expressions with high length, high depth and high subtraction ratio. No training or validation example longer than the allowed 21-character limit was introduced. This keeps the procedure consistent with the project constraints while increasing exposure to difficult structures near the boundary of the training distribution.

5 Results

The main results are reported in Table 1.

Model	Split	Exact-match accuracy	MAE	RMSE
CNN regressor	Validation	98.75%	0.0938	0.1425
CNN regressor	ID test	97.45%	0.1149	0.1839
CNN regressor	OOD test	89.15%	0.2271	0.4369
CNN regressor	Long test	61.07%	0.7118	1.3736
Transformer classifier	Validation	94.75%	0.0665	0.3775
Transformer classifier	ID test	94.20%	0.0700	0.3225
Transformer classifier	OOD test	77.60%	0.8415	2.8984
Transformer classifier	Long test	30.33%	3.9067	6.7313

Table 1: Main validation and test results.

The CNN regressor is the strongest model overall. It achieves 97.45% exact-match accuracy on the ID test split and remains highly robust on the OOD split, with 89.15% accuracy. Most importantly, it obtains 61.07% accuracy on the long-sequence test split, substantially above the required threshold. Its numerical errors are also low: RMSE remains below 0.5 on the OOD split and 1.38 on the long split.

The Transformer classifier performs well on validation and ID examples, reaching 94.20% ID accuracy. However, its degradation under distribution shift is much stronger. OOD accuracy drops to 77.60%, and long-sequence accuracy drops to 30.33%. Although this still exceeds the required long-sequence threshold, the gap between the two models is large on the hardest split.

A key difference between the models appears in MAE and RMSE. The CNN regressor has consistently lower numerical error on OOD and long examples. This suggests that even when the CNN is not exactly correct, its predictions often remain close to the true value. The Transformer classifier, instead, can make larger discrete class errors, which explains the much higher RMSE on shifted splits.

6 Generalization and Error Analysis

The main generalization challenge is the transition from ID examples to OOD and long-sequence examples. Validation and ID data have essentially the same structural range as the training set. By contrast, the OOD split contains longer, deeper and more operator-heavy formulas, while the long split contains expressions with lengths up to 27, depths up to 8 and operator counts up to 7. These values go beyond the support of the training distribution for several features.

The CNN regressor degrades much more gracefully across splits. Its accuracy falls from 97.45% on ID to 89.15% on OOD, a decrease of 8.30 percentage points. On the long split, accuracy decreases further to 61.07%, showing that length extrapolation remains difficult but not catastrophic. The corresponding increase in MAE from 0.1149 to 0.7118 indicates that long-sequence mistakes are usually still numerically moderate.

The Transformer classifier shows a sharper degradation. Its ID accuracy is 94.20%, but OOD accuracy decreases to 77.60%, and long-sequence accuracy decreases to 30.33%. The increase in RMSE is especially large: from 0.3225 on ID to 6.7313 on long examples. This suggests that the Transformer sometimes predicts classes far from the correct integer value when the expression structure moves away from the training distribution.

The long split differs from the training set along multiple dimensions. It is not only longer, but also deeper, more parenthesized and more subtraction-heavy. Therefore, poor long-sequence performance cannot be attributed to length alone. Parentheses change the effective order of operations, while subtraction introduces sign-sensitive and order-sensitive dependencies. These factors jointly increase the difficulty of extrapolation.

The results indicate that the CNN regressor learned a strong approximation of the arithmetic function within and moderately beyond the training distribution. Its success may be due to the combination of local convolutional filters, stable regression training and the custom rounding-aware loss. The Transformer classifier, although

theoretically well suited to global dependencies, appears more sensitive to distribution shift. This may be because classification treats output values as discrete labels, so a wrong prediction can jump to a distant class, increasing RMSE. It may also reflect the difficulty of training a Transformer to perform exact symbolic computation from limited short-sequence examples.

7 Conclusion

This project compared two neural approaches to learning an arithmetic calculator: a CNN regressor and a Transformer classifier. The CNN regressor achieved the best overall performance, with strong ID accuracy, high OOD robustness and the best long-sequence extrapolation. The Transformer classifier also performed well on validation and ID data, but generalized less effectively to longer and structurally shifted expressions.

The main finding is that, for this task, continuous convolutional regression was more robust than attention-based classification under the selected training protocol. However, this comparison does not prove that CNNs are universally superior to Transformers for symbolic arithmetic, because the two models differ both in architecture and output formulation. Rather, it shows that the CNN-regression approach was the most effective complete modelling strategy among those tested.

Future improvements could include more systematic data augmentation within the allowed 21-character training limit, additional analysis of error patterns by depth and subtraction ratio, and architectures with stronger structural biases for nested expressions.

Group Contribution Statement

All group members contributed to the project design, model implementation, experimental evaluation and final report writing. Specific contributions can be detailed here according to the actual work division.